Document number: P1976R2
Date: 2020-02-11
Audience: Library Working Group
Reply-to: *Tomasz Kamiński <tomaszkam at gmail dot com>*

Fixed-size span construction from dynamic range

1. Introduction

This paper provides more detailed explanation of PL250 NB issue. We explore issues with construction of fixed-size `span` construction from the range with the dynamic size. This constructor are source of the undefined behavior, without providing any synctatic suggestion on the user side.

To resolve the issues, we present three options:

- A. Separate fixed-size and dynamic `span` (remove fixed-size `span` for C++20)
- B. Disabling constructors
- C. Making constructors explicit (PL250 proposed resolution)

Per LEWG guidance in Belfast, the proposed resolution follows the option C (PL250 guidance) and marks the fixed-spize `span` constructors from dynamic-size range explicit.

```
                                                          Before              After

   void processFixed(std::span<int, 5>);
   void processDynamic(std::span<int>);

   std::vector<int> v3(3);
   std::vector<int> v5(5);


Dynamic range with different size (5 vs 3)


   processFixed(v3);                                      //     ill-formed             ill-formed
   processFixed({v3.data(), v3.data() + 3});              //     undefined-behavior     ill-formed

   processFixed(span<int, 5>(v3));                        //     ill-formed             undefined-behavior
   processFixed(span<int, 5>{v3.data(), v3.data() + 3});  //     undefined-behavior     undefined-behavior

   processFixed(span<int>(v3).first<5>());                //     undefined-behavior     undefined-behavior
   processFixed(span<int>{v3.data(), v3.data() + 3}.first<5>()); //     undefined-behavior     undefined-behavior

   span<int, 5> s = v3;                                   //     ill-formed             ill-formed
   span<int, 5> s(v3);                                    //     undefined-behavior     undefined-behavior

   span<int, 5> s = {v3.data(), v3.data() + 3};           //     undefined-behavior     ill-formed
   span<int, 5> s{v3.data(), v3.data() + 3};              //     undefined-behavior     undefined-behavior


Dynamic range with matching size (5 vs 5)


   processFixed(v5);                                      //     ill-formed             ill-formed
   processFixed({v5.data(), v5.data() + 5});              //     ok                     ill-formed

   processFixed(span<int, 5>(v5));                        //     ill-formed             ok
   processFixed(span<int, 5>{v5.data(), v5.data() + 5});  //     ok                     ok

   processFixed(span<int>(v5).first<5>());                //     ok                     ok
   processFixed(span<int>{v5.data(), v5.data() + 5}.first<5>()); //     ok                     ok

   span<int, 5> s = v5;                                   //     ill-formed             ill-formed
   span<int, 5> s(v5);                                    //     ok                     ok

   span<int, 5> s = {v5.data(), v5.data() + 5};           //     ok                     ill-formed
   span<int, 5> s{v5.data(), v5.data() + 5};              //     ok                     ok
```

2. Revision history

2.3. Revision 2

- Replaced iterator reference with array/range/OtherElementType in notes.

2.2. Revision 1

- Added wording for Option C.
- Included Tony Tables for proposed resolution (option C) in Introduction.
- Fixed minor typos in text.

2.1. Revision 0

Initial revision.

3. Motivation and Scope

The resolution of the LWG issue 3101 prevents user from running into accidental undefined-behavior when the `span` with fixed size is constructed from the range with the size that is not known at compile time. To illustrate:

```
void processFixed(std::span<int, 5>);

std::vector<int> v;
```

With the above declaration the following invocation is ill-formed:

```
processFixed(v); // ill-formed
```

Before the resolution of the issues, the above code was having undefined-behavior if the `v.size()` was different than `5` (size of `span` in declaration of `processFixed`).

However, the proposed resolution does not prevent the accidental undefined-behavior in situation when `(iterator, size)` or the `(iterator, sentinel)` constructor is used:

```
void processFixed({v.data(), v.size()}); // undefined-behavior if v.size() != 5
void processFixed({v.begin(), v.end()}); // undefined-behavior if v.size() != 5
```

3.2. Option A: Separate fixed-size and dynamic-size span (remove fixed-size span for C++20)

One of the option of resolving the issue is to separate the fixed-size and dynamic-size `span` into separate template. As it is to late for the C++20 for the introduction of the new template, such change would imply removal of the fixed-size `span` version of the span from the standard.

As consequence, the `span` template would become dynamicly sized, and would accept single type as template parameter:

```
template<class T> span;
```

Futhermore it would allow us to explore extending *`fixed-span`* construction to handle user-defined fixed-size ranges. Currently the standard regonizes only native arrays (`T[N]`), `std::array<T, N>` and fixed-size `std::span<T, N>` (where `N != std::dynamic_extent`) as fixed-size range. The appropariate trait was proposed in A SFINAE-friendly trait to determine the extent of statically sized containers.

3.2. Option B: Disabling constructors

We can follow the direction of the LWG issue 3101 and disable these constructor from particpating from the overload resolution entirelly. That would prevent the constructing the fixed-span from the dynamic range, and require the user to `first<N>()`/`last<N>`/`subspan<P, N>` methods explicitly.

```
void processFixed(std::span(v).first<5>());      // undefined-behavior if v.size() < 5
void processFixed(std::span(v).last<5>());       // undefined-behavior if v.size() < 5
void processFixed(std::span(v).subspan<1, 5>()); // undefined-behavior if v.size() < 6 = 1 + 5
```

[Note: Lack of template parameter for `span` in above examples is intentional - they use deduction guides.]

Tony Tables for option B.

Before		**After: Option B**
`void processFixed(std::span<int, 5>);` `void processDynamic(std::span<int>);`		
Dynamic range with different size		
`std::vector<int> v3(3);`		
`processFixed(v3);`	`// ill-formed`	`processFixed(v3);`
`processFixed({v3.data(), v3.data() + 3});`	`// undefined-behavior`	`processFixed({v3.data(), v3.data() + 3});`
`processFixed({v3.data(), 3});`	`// undefined-behavior`	`processFixed({v3.data(), 3});`
`processFixed(span<int, 5>(v3));`	`// ill-formed`	`processFixed(span<int, 5>(v3));`
`processFixed(span<int, 5>{v3.data(), v3.data() + 3});`	`// undefined-behavior`	`processFixed(span<int, 5>{v3.data(), v3.data() +`
`processFixed(span<int, 5>{v3.data(), 3});`	`// undefined-behavior`	`processFixed(span<int, 5>{v3.data(), 3});`
`processFixed(span<int>(v3).first<5>());`	`// undefined-behavior`	`processFixed(span<int>(v3).first<5>());`
`processFixed(span<int>{v3.data(), v3.data() + 3}.first<5>());`	`// undefined-behavior`	`processFixed(span<int>{v3.data(), v3.data() + 3}.`
`processFixed(span<int>{v3.data(), 3}.first<5>());`	`// undefined-behavior`	`processFixed(span<int>{v3.data(), 3}.first<5>());`
Dynamic range with matching size		
`std::vector<int> v5(5);`		
`processFixed(v5);`	`// ill-formed`	`processFixed(v5);`
`processFixed({v5.data(), v5.data() + 5});`	`// ok`	`processFixed({v5.data(), v5.data() + 5});`
`processFixed({v5.data(), 5});`	`// ok`	`processFixed({v5.data(), 5});`
`processFixed(span<int, 5>(v5));`	`// ill-formed`	`processFixed(span<int, 5>(v5));`
`processFixed(span<int, 5>{v5.data(), v5.data() + 5});`	`// ok`	`processFixed(span<int, 5>{v5.data(), v5.data() +`
`processFixed(span<int, 5>{v5.data(), 5});`	`// ok`	`processFixed(span<int, 5>{v5.data(), 5});`
`processFixed(span<int>(v5).first<5>());`	`// ok`	`processFixed(span<int>(v5).first<5>());`
`processFixed(span<int>{v5.data(), v5.data() + 5}.first<5>());`	`// ok`	`processFixed(span<int>{v5.data(), v5.data() + 5}.`
`processFixed(span<int>{v5.data(), 5}.first<5>());`	`// ok`	`processFixed(span<int>{v5.data(), 5}.first<5>());`

3.3. Option C: Making constructors explicit

This is original resolution proposed in PL250.

The construction of the fixed-sized `span` from the dynamicly sized range, is not indentity operation - this operation assumes additional semantic property of the type (size of the range). Such conversion between semantically different types, should not be implicit. We can resolve the problem, by marking all of such constructor explicit, as follows:

Destination/Source	**Fixed**	**Dynamic**
Fixed	implicit (ill-formed if source.size() != dest.size())	explicit (undefined-behavior if source.size() != dest.size())
Dynamic	implicit (always ok)	implicit (always ok)

Tony Tables for option C.

Before

```
void processFixed(std::span<int, 5>);
void processDynamic(std::span<int>);
```

Dynamic range with different size

```
std::vector<int> v3(3);
processFixed(v3);                                       // ill-formed
processFixed({v3.data(), v3.data() + 3});               // undefined-behavior
processFixed({v3.data(), 3});                           // undefined-behavior

processFixed(span<int, 5>(v3));                         // ill-formed
processFixed(span<int, 5>{v3.data(), v3.data() + 3});   // undefined-behavior
processFixed(span<int, 5>{v3.data(), 3});               // undefined-behavior

processFixed(span<int>(v3).first<5>());                 // undefined-behavior
processFixed(span<int>{v3.data(), v3.data() + 3}.first<5>()); // undefined-behavior
processFixed(span<int>{v3.data(), 3}.first<5>());       // undefined-behavior
```

Dynamic range with matching size

```
std::vector<int> v5(5);
processFixed(v5);                                       // ill-formed
processFixed({v5.data(), v5.data() + 5});               // ok
processFixed({v5.data(), 5});                           // ok

processFixed(span<int, 5>(v5));                         // ill-formed
processFixed(span<int, 5>{v5.data(), v5.data() + 5});   // ok
processFixed(span<int, 5>{v5.data(), 5});               // ok

processFixed(span<int>(v5).first<5>());                 // ok
processFixed(span<int>{v5.data(), v5.data() + 5}.first<5>()); // ok
processFixed(span<int>{v5.data(), 5}.first<5>());       // ok
```

After: Option C

```
processFixed(v3);
processFixed({v3.data(), v3.data() + 3});
processFixed({v3.data(), 3});

processFixed(span<int, 5>(v3));
processFixed(span<int, 5>{v3.data(), v3.data() +
processFixed(span<int, 5>{v3.data(), 3});

processFixed(span<int>(v3).first<5>());
processFixed(span<int>{v3.data(), v3.data() + 3}.
processFixed(span<int>{v3.data(), 3}.first<5>());
```

```
processFixed(v5);
processFixed({v5.data(), v5.data() + 5});
processFixed({v5.data(), 5});

processFixed(span<int, 5>(v5));
processFixed(span<int, 5>{v5.data(), v5.data() +
processFixed(span<int, 5>{v5.data(), 5});

processFixed(span<int>(v5).first<5>());
processFixed(span<int>{v5.data(), v5.data() + 5}.
processFixed(span<int>{v5.data(), 5}.first<5>());
```

3.4. No impact on dynamic-sized span

All proposed options (including removal) does not have any impact on the construction of the dynamic-sized span (i.e. `span<T>`). The construction changes affect only cases when `N != std::dynamic_extent`.

3.5. Option B vs C: Impact on initialization

The major difference between the option B and option C, is the impact the impact on the initialization of the span variables. Some of the readers, may consider the difference between various syntaxes and their meaning two subtle.

Tony Tables for initialization.

Option B

```
std::vector<int> v3(3);

span<int, 5> s = v3;                            // ill-formed
span<int, 5> s(v3);                             // ill-formed
auto s = span<int, 5>(v3);                      // ill-formed

span<int, 5> s = {v3.data(), v3.data() + 3};    // ill-formed
span<int, 5> s{v3.data(), v3.data() + 3};       // ill-formed
auto s = span<int, 5>{v3.data(), v3.data() + 3}; // ill-formed
```

Option C

```
span<int, 5> s = v3;                            // ill-formed
span<int, 5> s(v3);                             // undefined-behavior
auto s = span<int, 5>(v3);                      // undefined-behavior

span<int, 5> s = {v3.data(), v3.data() + 3};    // ill-formed
span<int, 5> s{v3.data(), v3.data() + 3};       // undefined-behavior
auto s = span<int, 5>{v3.data(), v3.data() + 3}; // undefined-behavior
```

3.6. Option B and C: Construction from fixed-sized range

Neither option B nor C, proposes any change to the behavior of the construction of the fixed-size `span` from the ranges that are recognized by the standard as fixed-size: native arrays (`T[N]`), `std::array<T, N>` and fixed-size `std::span<T, N>` (where `N != std::dynamic_extent`). The construction is implicit if size of the source is the same as the size of destination, ill-formed otherwise.

```
void processFixed(span<int, 5>);

std::array<int, 3> a3;
std::array<int, 5> a5;

processFixed(a3); // ill-formed
```

```
processFixed(a5); // ok

std::span<int, 3> s3(a3);
std::span<int, 5> s5(a5);

processFixed(s3); // ill-formed
processFixed(s5); // ok
```

3.7. Option B and C: Impact of the P1394

The P1394: Range constructor for `std::span` (that is targeting C++20) generalized the constructor of the span.

The `Container` constructor was replaced with the `Range` constructor, that have the same constrain (i.e. it is disabled for fixed-size `span`), so the original example remain ill-formed:

```
processFixed(v); // ill-formed
```

In addition it replaces the `(pointer, size)` and `(pointer, pointer)` constructor, with more general `(iterator, size)` and `(iterator, sentinel)`. As consequence in addition the undefined-behavior is exposed in more situations:

```
void processFixed({v.begin(), v.size()}); // undefined-behavior if v.size() != 5
void processFixed({v.begin(), v.end()});  // undefined-behavior if v.size() != 5
```

in addition to:

```
void processFixed({v.data(), v.size()});            // undefined-behavior if v.size() != 5
void processFixed({v.data(), v.data() + v.size()}); // undefined-behavior if v.size() != 5
```

Changes presented in this paper still apply after signature changes from P1394.

4. Impact and Implementability

As the `std:span` was introduced in C++20, the changes introduce in these paper (regardless of the selected option) cannot break existing code. In addition, all presented options do not affect uses of `span` with the dynamic size.

The implementation of the option B requires duplicating a constrain:
Constrains: `extent == dynamic_extent` is `true`.
that is already present in `Container`/`Range` constructor ([span.cons] p14.1) to 3 additional constuctors. In can be implemented using the SFINAE tricks (`std::enable_if`) or `requires` clause.

The implementation of the option C mostly requires adding an conditional explicit specifier to 4 constuctors:

```
explicit(extent != dynamic_extent)
```

5. Proposed Wording

The proposed wording changes refer to N4842 (C++ Working Draft, 2019-11-27).

Apply following changes to section [span.overview] Overview:

```
// [span.cons], constructors, copy, and assignment
constexpr span() noexcept;
template<class It>
  constexpr explicit(extent != dynamic_extent) span(It first, size_type count);
template<class It, class End>
  constexpr explicit(extent != dynamic_extent) span(It first, End last);
template<size_t N>
  constexpr span(element_type (&arr)[N]) noexcept;
template<size_t N>
  constexpr span(array<value_type, N>& arr) noexcept;
template<size_t N>
  constexpr span(const array<value_type, N>& arr) noexcept;
template<class R>
  constexpr explicit(extent != dynamic_extent) span(R&& r);
constexpr span(const span& other) noexcept = default;
template<class OtherElementType, size_t OtherExtent>
  constexpr explicit(see below) span(const span<OtherElementType, OtherExtent>& s) noexcept;
```

Apply following changes to section [span.cons] Constructors, copy, and assignment:

```
template<class It>
  constexpr explicit(extent != dynamic_extent) span(It first, size_type count);
```

Constraints: Let `U` be `remove_reference_t<iter_reference_t<It>>`.

- `It` satisfies `contiguous_iterator`.
- `is_convertible_v<U(*)[], element_type(*)[]>` is `true`. [Note: The intent is to allow only qualification conversions of the iterator reference type to `element_type`. — end note]

Preconditions:

- `[first, first + count)` is a valid range.
- `It` models `contiguous_iterator`.
- If `extent` is not equal to `dynamic_extent`, then `count` is equal to `extent`.

Effects: Initializes `data_` with `to_address(first)` and `size_` with `count`.

Throws: When and what `to_address(first)` throws.

```
template<class It, class End>
  constexpr explicit(extent != dynamic_extent) span(It first, End last);
```

Constraints: Let `U` be `remove_reference_t<iter_reference_t<It>>`.

- `is_convertible_v<U(*)[], element_type(*)[]>` is `true`. [Note: The intent is to allow only qualification conversions of the iterator reference type to `element_type`. — end note]
- `It` satisfies `contiguous_iterator`.
- `End` satisfies `sized_sentinel_for<It>`.
- `is_convertible_v<End, size_t>` is `false`.

Preconditions:

- If `extent` is not equal to `dynamic_extent`, then `last - first` is equal to `extent`.
- `[first, first + count)` is a valid range.
- `It` models `contiguous_iterator`.
- `End` models `sized_sentinel_for<It>`.

Effects: Initializes `data_` with `to_address(first)` and `size_` with `last - first`.

Throws: When and what `to_address(first)` throws.

```
template<size_t N> constexpr span(element_type (&arr)[N]) noexcept;
template<size_t N> constexpr span(array<value_type, N>& arr) noexcept;
template<size_t N> constexpr span(const array<value_type, N>& arr) noexcept;
```

Constraints: Let `U` be `remove_pointer_t<decltype(data(arr))>`.

- `extent == dynamic_extent || N == extent` is `true`, and
- ~~`remove_pointer_t<decltype(data(arr))>(*)[] is convertible to ElementType(*)[]`.~~
- `is_convertible_v<U(*)[], element_type(*)[]>` is `true`. [Note: The intent is to allow only qualification conversions of the array element type to `element_type`. — end note]

Effects: Constructs a `span` that is a view over the supplied array.

Postconditions: `size() == N && data() == data(arr)`.

```
template<class R>
  constexpr explicit(extent != dynamic_extent) span(R&& r);
```

Constraints: Let `U` be `remove_reference_t<ranges::range_reference_t<R>>`.

- ~~`extent == dynamic_extent is true`.~~
- `R` satisfies `ranges::contiguous_range` and `ranges::sized_range`.
- Either `R` satisfies `safe_range` or `is_const_v<element_type>` is `true`.
- `remove_cvref_t<R>` is not a specialization of `span`.
- `remove_cvref_t<R>` is not a specialization of `array`.
- `is_array_v<remove_cvref_t<R>>` is `false`.
- `is_convertible_v<U(*)[], element_type(*)[]>` is `true`. [Note: The intent is to allow only qualification conversions of the ~~iterator~~range reference type to `element_type`. — end note]

Preconditions:

- If `extent` is not equal to `dynamic_extent`, then `ranges::size(r)` is equal to `extent`.
- `R` models `ranges::contiguous_range` and `ranges::sized_range`.
- If `is_const_v<element_type>` is `false`, `R` models `safe_range`.

Effects: Initializes `data_` with `ranges::data(r)` and `size_` with `ranges::size(r)`.

Throws: What and when `ranges::data(r)` and `ranges::size(r)` throws.

[...]

```
template<class OtherElementType, size_t OtherExtent>
  constexpr explicit(see below) span(const span<OtherElementType, OtherExtent>& s) noexcept;
```

Constraints:

- ~~`Extent`~~`extent == dynamic_extent || OtherExtent == dynamic_extent || ~~`Extent`~~`extent == OtherExtent` is `true`, and
- ~~`OtherElementType(*)[] is convertible to ElementType(*)[]`.~~
- `is_convertible_v<OtherElementType(*)[], element_type(*)[]>` is `true`. [Note: The intent is to allow only qualification conversions of the `OtherElementType` to `element_type`. — end note]

Preconditions: If `extent` is not equal to `dynamic_extent`, then `s.size()` is equal to `extent`.

Effects: Constructs a `span` that is a view over the range `[s.data(), s.data() + s.size())`.

Postconditions: `size() == other.size() && data() == other.data()`

Remarks: The expression inside `explicit` is equivalent to: `extent != dynamic_extent && OtherExtent == dynamic_extent`.

Update the value of the `__cpp_lib_span` in [version.syn] Header `<version>` synopsis to reflect the date of approval of this proposal.

6. Acknowledgements

Andrzej Krzemieński, Casey Carter, Tim Song and Jeff Garland offered many useful suggestions and corrections to the proposal.

Special thanks and recognition goes to Sabre (http://www.sabre.com) for supporting the production of this proposal and author's participation in standardization committee.

7. References

1. Poland, "PL250 22.07.3.2 [span.cons] size mismatch for fixed-sized span", (PL250, https://github.com/cplusplus/nbballot/issues/246)
2. Stephan T. Lavavej, "span's Container constructors need another constraint", (LWG3103, https://wg21.link/lwg3103)
3. Corentin Jabot, Casey Carter, "Range constructor for `std::span`", (P1394R4, https://wg21.link/p1394r4)
4. Corentin Jabot, Casey Carter, "A SFINAE-friendly trait to determine the extent of statically sized containers" (P1419R0, https://wg21.link/p1419r0)
5. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4842, https://wg21.link/n4842)